DESARROLLO WEB BACKEND

UNIDAD 1

Fundamentos del Desarrollo Web

Material de Estudio Teórico

CONTENIDO DE LA UNIDAD

1.1 Conceptos generales del desarrollo web
1.2 Diferencias entre Front End y Back End
1.3 Rol e importancia del Backend en una aplicación web
1.4 Protocolo HTTP
1.4.1 Métodos HTTP
1.4.2 Cabeceras HTTP
1.4.3 Códigos de estado
1.5 Arquitecturas de aplicaciones web
1.5.1 Arquitectura monolítica y patrón MVC
1.5.2 Arquitectura de microservicios
1.5.3 Comparativa y criterios de selección

OBJETIVO DE LA UNIDAD

Al finalizar esta unidad, el estudiante comprenderá los fundamentos del desarrollo web moderno, distinguirá las responsabilidades del front-end y el back-end, dominará el protocolo HTTP como base de la comunicación web, y evaluará las principales arquitecturas de aplicación para seleccionar la más adecuada según el contexto de cada proyecto.

1.1 Conceptos Generales del Desarrollo Web

El desarrollo web engloba el conjunto de disciplinas, técnicas y herramientas necesarias para construir, publicar y mantener sitios y aplicaciones accesibles a través de Internet o redes privadas. Lejos de ser una tecnología estática, ha evolucionado de manera acelerada desde sus orígenes en los años noventa hasta convertirse en el pilar de la economía digital actual.

DEFINICIÓN — APLICACIÓN WEB

Una aplicación web es un programa de software que se ejecuta en un servidor remoto y es accedido por los usuarios a través de un navegador web (cliente). A diferencia de las aplicaciones de escritorio, no requiere instalación local: el navegador recibe e interpreta los resultados que el servidor procesa y envía.

Componentes fundamentales de una arquitectura web

- **Cliente (navegador)**
  - Software que el usuario utiliza para acceder a la aplicación. Interpreta HTML, CSS y JavaScript para renderizar la interfaz.
- **Servidor web**
  - Recibe solicitudes del cliente y devuelve respuestas. Ejemplos: Apache, Nginx, IIS.
- **Servidor de aplicaciones**
  - Ejecuta la lógica de negocio y genera contenido dinámico.
- **Base de datos**
  - Sistema de almacenamiento persistente donde se guardan y recuperan los datos de la aplicación.
- **Red / Infraestructura**
  - Conecta a clientes y servidores mediante protocolos estándar como TCP/IP y HTTP/HTTPS.

Evolución de la Web

Generación	Período	Características principales
Web 1.0	1991 – 2004	Páginas estáticas, solo lectura. Contenido generado exclusivamente por los propietarios del sitio.
Web 2.0	2004 – Presente	Interactividad, participación del usuario, redes sociales, AJAX, APIs REST. Contenido generado por usuarios.
Web 3.0	Emergente	Web semántica, descentralización (blockchain), IA integrada, mayor privacidad y propiedad de datos.

DATO RELEVANTE

Actualmente más del 60 % del tráfico web proviene de dispositivos móviles. Esto hace que el diseño responsivo (Responsive Design) y la optimización de rendimiento sean requisitos, no opciones.

1.2 Diferencias entre Front End y Back End

El desarrollo web moderno se divide en dos grandes especialidades complementarias. Aunque ambas trabajan juntas para producir una aplicación funcional, cada una opera en entornos distintos, utiliza tecnologías diferentes y persigue objetivos propios.

Dimensión	Front End	Back End
¿Qué es?	Todo lo que el usuario ve e interactúa en el navegador.	Lógica del servidor, bases de datos y servicios que operan detrás de escena.
Entorno de ejecución	Navegador del usuario (cliente).	Servidor (nube, on-premise, contenedores Docker/K8s).
Tecnologías clave	HTML, CSS, JavaScript, React, Angular, Vue, Tailwind.	Node.js, Python, Java, PHP, Ruby, Go, .NET, bases de datos SQL/NoSQL.
Responsabilidad	Presentación, experiencia de usuario (UX/UI), accesibilidad, SEO.	Lógica de negocio, seguridad, gestión de datos, APIs, autenticación.
Métricas clave	Tiempo de carga, usabilidad, compatibilidad entre navegadores.	Tiempo de respuesta (latencia), disponibilidad (uptime), escalabilidad.

FULL STACK DEVELOPMENT

Un desarrollador Full Stack domina ambas capas y puede construir una aplicación completa de manera autónoma. Es especialmente valioso en startups y equipos reducidos. Sin embargo, en organizaciones medianas y grandes suele preferirse la especialización para maximizar la profundidad técnica en cada área.

1.3 Rol e Importancia del Backend en una Aplicación Web

El backend es el núcleo funcional de cualquier aplicación web. Sin él, el frontend sería simplemente una interfaz estática sin datos reales ni capacidad de procesamiento. El backend garantiza que la aplicación sea segura, eficiente, escalable y confiable.

Responsabilidades del desarrollador Backend

- Gestión de la lógica de negocio — implementa las reglas que gobiernan cómo se procesan, validan y transforman los datos.
- Autenticación y autorización — controla quién accede al sistema y qué acciones puede realizar cada rol de usuario.
- Gestión de bases de datos — diseña el modelo de datos y ejecuta operaciones CRUD con integridad y eficiencia.
- Exposición de APIs — publica interfaces (REST, GraphQL, gRPC) que el frontend u otros servicios consumen.

- Procesamiento asíncrono — gestiona tareas diferidas (emails, reportes, procesamiento de imágenes) mediante colas de mensajes.
- Seguridad — implementa cifrado TLS, protección contra inyecciones SQL, XSS, CSRF y otras vulnerabilidades OWASP.
- Caché y rendimiento — usa sistemas como Redis o Memcached para reducir la carga en la base de datos.

EJEMPLO — FLUJO DE UNA COMPRA EN LÍNEA

Cuando un usuario presiona 'Comprar', el backend ejecuta en milisegundos: (1) verificar stock en tiempo real, (2) comunicarse con la pasarela de pago, (3) aplicar descuentos y calcular impuestos, (4) registrar la transacción en la base de datos, (5) actualizar el inventario, (6) generar la factura y (7) enviar el email de confirmación. Todo de forma transaccional: si algún paso falla, la operación completa se revierte.

1.4 Protocolo HTTP

DEFINICIÓN — HTTP (HYPERTEXT TRANSFER PROTOCOL)

Es el protocolo de comunicación fundamental de la Web. Define las reglas bajo las cuales clientes y servidores intercambian información. Es un protocolo sin estado (stateless): cada solicitud es independiente y el servidor no retiene información de solicitudes anteriores por defecto.

La comunicación HTTP sigue siempre el mismo patrón: el cliente envía una solicitud (Request) y el servidor devuelve una respuesta (Response). Cada uno de estos mensajes tiene una estructura bien definida.

Ciclo de vida de una solicitud HTTP

1. El usuario ingresa una URL o la aplicación dispara una solicitud programáticamente.
2. El navegador resuelve el dominio vía DNS para obtener la dirección IP del servidor.
3. Se establece una conexión TCP/IP (y TLS/SSL si el esquema es HTTPS).
4. El cliente envía la solicitud HTTP con: método, ruta, cabeceras y opcionalmente un cuerpo.
5. El servidor procesa la solicitud, ejecuta la lógica correspondiente y genera una respuesta.
6. La respuesta llega al cliente con: código de estado, cabeceras y el cuerpo de la respuesta.

HTTP VS HTTPS — DIFERENCIA CRÍTICA

HTTPS agrega una capa de seguridad mediante TLS (Transport Layer Security), que cifra toda la comunicación entre cliente y servidor. Hoy en día es el estándar obligatorio: los navegadores modernos marcan como 'No seguras' las páginas HTTP, y Google penaliza en posicionamiento SEO a los sitios que no lo implementen.

Estructura de una solicitud HTTP

```
POST /api/usuarios HTTP/1.1
Host: api.ejemplo.com
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Accept: application/json

{
  "nombre": "Ana García",
  "email": "ana@ejemplo.com",
  "password": "s3cur3P@ss"
}
```

Estructura de una respuesta HTTP

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=UTF-8
Location: /api/usuarios/99
X-Request-Id: a1b2c3d4

{
  "id": 99,
  "nombre": "Ana García",
  "email": "ana@ejemplo.com",
  "creado_en": "2026-03-04T10:30:00Z"
}
```

1.4.1 Métodos HTTP

Los métodos HTTP, también llamados verbos, indican la acción que el cliente desea realizar sobre un recurso. Cada método tiene semántica precisa que debe respetarse para construir APIs coherentes, predecibles y conformes con el estándar REST.

Método	Descripción y uso
GET	Recupera un recurso sin modificarlo. Es seguro (no altera estado del servidor) e idempotente (el mismo resultado siempre). Ej: GET /productos/42
POST	Crea un nuevo recurso o envía datos para ser procesados. No es idempotente: dos llamadas idénticas pueden crear dos recursos distintos. Ej: POST /productos
PUT	Reemplaza completamente un recurso existente. Es idempotente: enviar la misma solicitud varias veces produce el mismo estado final. Ej: PUT /productos/42
PATCH	Modifica parcialmente un recurso: solo actualiza los campos enviados en el cuerpo. Ej: PATCH /productos/42 { "precio": 29.99 }
DELETE	Elimina un recurso. Es idempotente: eliminar un recurso inexistente suele retornar 404 o 204, pero no cambia el estado. Ej: DELETE /productos/42
HEAD	Idéntico a GET pero sin cuerpo de respuesta. Útil para verificar la existencia de un recurso o sus metadatos sin descargar el contenido.
OPTIONS	Devuelve los métodos permitidos para un recurso. Usado en el mecanismo CORS preflight que los navegadores ejecutan antes de solicitudes cross-origin.

Método	¿Seguro?	¿Idempotente?	Cuerpo en request	Uso principal
GET	Sí	Sí	No	Leer / consultar recursos
POST	No	No	Sí	Crear recursos, login, acciones
PUT	No	Sí	Sí	Reemplazar recurso completo
PATCH	No	No*	Sí	Actualización parcial
DELETE	No	Sí	Opcional	Eliminar recurso
HEAD	Sí	Sí	No	Verificar existencia / metadata
OPTIONS	Sí	Sí	No	CORS preflight, introspección

** PATCH puede ser idempotente si la operación es determinista, pero no se garantiza por especificación.*

1.4.2 Cabeceras HTTP

Las cabeceras (headers) son pares clave-valor que acompañan a las solicitudes y respuestas HTTP. Transportan metadatos esenciales sobre la comunicación sin formar parte del contenido principal: tipo de datos, autenticación, configuración de caché, información del cliente, políticas de seguridad, entre otros.

Cabeceras de Solicitud (Request Headers)

Cabecera	Ejemplo de valor	Propósito
Content-Type	application/json	Indica el formato del cuerpo de la solicitud.
Accept	application/json	Formatos que el cliente acepta en la respuesta.
Authorization	Bearer eyJhbGci...	Credenciales de autenticación (JWT, API Key, Basic).
User-Agent	Mozilla/5.0 (Windows...)	Identifica el software cliente: navegador, app móvil, bot.
Accept-Language	es-ES, en;q=0.9	Idiomas preferidos por el cliente (para i18n).
Cache-Control	no-cache	Directivas para el comportamiento de la caché local.
Origin	https://app.miempresa.com	Origen de la solicitud; clave en el flujo CORS.
Cookie	session=abc123; theme=dark	Cookies persistentes que el navegador envía automáticamente.

Cabeceras de Respuesta (Response Headers)

Cabecera	Ejemplo de valor	Propósito
Content-Type	application/json; charset=UTF-8	Formato y codificación del cuerpo de la respuesta.
Set-Cookie	token=xyz; HttpOnly; Secure; SameSite=Strict	Establece cookies en el cliente con atributos de seguridad.
Access-Control-Allow-Origin	https://app.miempresa.com	CORS: indica qué orígenes pueden acceder al recurso.
Cache-Control	max-age=3600, public	Indica por cuánto tiempo puede cachearse la respuesta.
Location	/api/usuarios/99	URL del recurso recién creado (usada con 201 Created).
X-RateLimit-Remaining	45	Solicitudes restantes antes de activar el rate limiting.
Strict-Transport-Security	max-age=31536000; includeSubDomains	Fuerza el uso de HTTPS (HTTP Strict Transport Security).

CABECERAS PERSONALIZADAS (X-HEADERS)

Es una práctica común agregar cabeceras personalizadas con el prefijo X- para información de la aplicación (ej: X-Request-Id, X-Tenant-Id). Aunque el prefijo X- fue deprecado formalmente por RFC 6648, sigue siendo ampliamente utilizado en la industria. Úsalas para datos que no encajan en cabeceras estándar, pero documenta su contrato claramente.

1.4.3 Códigos de Estado HTTP

Los códigos de estado son números de tres dígitos que el servidor incluye en cada respuesta para indicar el resultado del procesamiento. Se agrupan en cinco familias según el primer dígito, cada una con semántica propia que el cliente puede interpretar programáticamente.

Código	Nombre	Descripción
2xx	**Éxito — La solicitud fue recibida, comprendida y aceptada correctamente.**	
200	**OK**	Solicitud exitosa. La respuesta contiene el recurso solicitado (GET, PUT, PATCH).
201	**Created**	Recurso creado exitosamente. La cabecera Location indica la URL del nuevo recurso.
204	**No Content**	Éxito sin cuerpo de respuesta. Típico en DELETE exitosos o PUT sin retorno.
3xx	**Redirección — El cliente debe realizar una acción adicional para completar la solicitud.**	
301	**Moved Permanently**	Redirección permanente. Los motores de búsqueda actualizan sus índices.
302	**Found**	Redirección temporal. El cliente debe seguir usando la URL original.

Código	Nombre	Descripción
304	**Not Modified**	El recurso no cambió desde la última vez. El cliente usa su caché local.
4xx	**Error del cliente — La solicitud tiene un error que el cliente debe corregir.**	
400	**Bad Request**	Solicitud malformada o con datos inválidos (sintaxis incorrecta).
401	**Unauthorized**	Autenticación requerida o credenciales inválidas / expiradas.
403	**Forbidden**	Autenticado pero sin permisos para el recurso solicitado.
404	**Not Found**	El recurso solicitado no existe en el servidor.
409	**Conflict**	Conflicto con el estado actual del recurso (ej: email duplicado).
422	**Unprocessable Entity**	Datos semánticamente inválidos: falla validación de negocio.
429	**Too Many Requests**	Rate limiting activado: demasiadas solicitudes en el período definido.
5xx	**Error del servidor — El servidor falló al procesar una solicitud válida.**	
500	**Internal Server Error**	Error genérico del servidor: bug no manejado, excepción no capturada.
502	**Bad Gateway**	El servidor actuó como proxy y recibió respuesta inválida del upstream.
503	**Service Unavailable**	El servidor no puede atender solicitudes: sobrecarga o mantenimiento.
504	**Gateway Timeout**	El servidor proxy no recibió respuesta del upstream a tiempo.

REGLA PRÁCTICA PARA APIS REST

Utiliza siempre el código de estado más específico disponible. Retornar 200 OK para un recurso no encontrado o 500 para un error de validación es una mala práctica que dificulta el diagnóstico y rompe los contratos de las APIs. Los clientes bien escritos toman decisiones basándose en los códigos de estado, no en el cuerpo de la respuesta.

1.5 Arquitecturas de Aplicaciones Web

La arquitectura de software define cómo se organizan y comunican los componentes de un sistema. Es una de las decisiones más estratégicas en el desarrollo de software: afecta la escalabilidad, la mantenibilidad, los costos operativos y la velocidad del equipo. No existe una arquitectura universalmente superior; la elección correcta depende del contexto.

1.5.1 Arquitectura Monolítica y Patrón MVC

DEFINICIÓN — ARQUITECTURA MONOLÍTICA

En una arquitectura monolítica, todos los componentes de la aplicación — interfaz de usuario, lógica de negocio y acceso a datos — están integrados en un único proceso

desplegable. Se distribuye como un solo artefacto (JAR, WAR, binario, etc.) y comparte un único proceso y espacio de memoria.

El Patrón MVC (Model – View – Controller)

MVC es la forma más extendida de organizar el código dentro de un monolito. Separa la aplicación en tres capas con responsabilidades bien definidas, reduciendo el acoplamiento y facilitando el mantenimiento.

Capa	Responsabilidad	Ejemplos concretos
Model (Modelo)	Representa los datos y la lógica de negocio. Interactúa con la base de datos. Valida reglas de dominio.	Clases de entidades, ORM (Eloquent, Hibernate, Sequelize), repositorios, servicios de dominio.
View (Vista)	Presenta los datos al usuario. Genera HTML dinámico o serializa la respuesta en JSON/XML para APIs.	Plantillas Blade/Twig/Thymeleaf, componentes React/Vue, serializadores JSON.
Controller (Controlador)	Recibe las solicitudes HTTP, coordina el Model y la View, y retorna la respuesta al cliente. Es el 'director de orquesta'.	Clases Controller, Route Handlers, Middlewares de validación de entrada.

Ventajas y Desventajas

	Ventajas	Desventajas
Desarrollo	Simple de iniciar; una sola base de código; debug más directo.	A medida que crece, el código se vuelve difícil de mantener.
Despliegue	Un solo artefacto; proceso de CI/CD simple.	Escalar requiere replicar toda la aplicación, no solo el cuello de botella.
Operación	Menor complejidad operativa; sin latencia de red entre capas.	Un bug crítico puede derribar toda la aplicación.
Equipo	Ideal para equipos pequeños (1-8 personas).	Dificulta el trabajo paralelo de equipos grandes; conflictos frecuentes.

¿CUÁNDO ELEGIR UN MONOLITO?

Para proyectos nuevos, MVPs, startups o cuando el dominio no está bien definido. Un monolito bien estructurado puede soportar millones de usuarios (StackOverflow, Shopify y GitHub fueron monolitos durante años). La clave es mantener el código modular internamente desde el principio.

1.5.2 Arquitectura de Microservicios

DEFINICIÓN — MICROSERVICIOS

La arquitectura de microservicios divide la aplicación en servicios pequeños, independientes y desplegables por separado. Cada servicio es responsable de una capacidad de negocio

específica, tiene su propio proceso, su propia base de datos y su ciclo de vida de despliegue independiente.

Principios fundamentales

- Responsabilidad única — cada servicio hace una sola cosa y la hace bien (SRP a escala de servicio).
- Autonomía — los servicios se despliegan, escalan y actualizan de forma independiente sin afectar al resto.
- Comunicación vía APIs — los servicios se comunican mediante HTTP/REST, gRPC o mensajería asíncrona (Kafka, RabbitMQ).
- Descentralización de datos — cada servicio gestiona su propia base de datos, eligiendo el motor más adecuado para sus necesidades.
- Resiliencia — el fallo de un servicio no debe propagarse. Se implementan patrones como Circuit Breaker, Retry y Timeout.

Patrones de comunicación entre servicios

Tipo	Tecnologías comunes	Cuándo usarlo
Síncrona	HTTP/REST, gRPC	Cuando el llamador necesita la respuesta de inmediato para continuar su flujo.
Asíncrona	Kafka, RabbitMQ, AWS SQS	Tareas que pueden procesarse en diferido; desacopla productores de consumidores.
Event-Driven	Event Bus, Webhooks	Notificación de cambios de estado entre servicios sin acoplamiento directo.

Ventajas y Desventajas

	Ventajas	Desventajas
Escalabilidad	Escala servicios individuales según demanda real.	Requiere infraestructura de orquestación compleja (Kubernetes).
Despliegue	Deployments independientes y frecuentes por equipo.	Más difícil de testear de extremo a extremo; requiere contratos de API.
Tecnología	Cada servicio puede usar el lenguaje y BD más adecuados.	Heterogeneidad puede dificultar la incorporación de nuevos miembros.
Resiliencia	Un servicio caído no necesariamente derriba la aplicación.	Fallos en red y consistencia eventual son más difíciles de gestionar.
Equipo	Equipos autónomos por servicio; mayor velocidad de entrega.	Exige madurez en DevOps, observabilidad (logs, métricas, tracing).

NO EMPIECES CON MICROSERVICIOS

Martin Fowler y otros expertos advierten: 'Monolith First'. Los microservicios resuelven problemas de escala organizacional y técnica que los equipos pequeños aún no tienen.

> Comenzar con microservicios agrega una complejidad operativa enorme que puede hundir un proyecto antes de que demuestre valor de negocio.

1.5.3 Comparativa y Criterios de Selección

La elección entre arquitecturas no es binaria ni permanente. Muchos sistemas exitosos comienzan como monolitos modulares y extraen microservicios de forma incremental cuando la necesidad es demostrable con datos de producción.

Criterio de decisión	Favorece Monolito	Favorece Microservicios
Tamaño del equipo	1 – 10 personas	Múltiples squads autónomos (+10 personas)
Madurez del dominio	Dominio poco definido o en exploración	Dominio estable y bien comprendido
Tiempo al mercado	Prioridad máxima (MVP, startup)	Puede invertirse en infraestructura inicial
Requisitos de escala	Escala uniforme en todo el sistema	Partes del sistema escalan de forma diferente
Complejidad operativa	Equipo sin experiencia en DevOps	Equipo con madurez en K8s, CI/CD, observabilidad
Presupuesto de infra	Limitado o ajustado	Presupuesto para orquestación y herramientas
Frecuencia de cambio	El sistema cambia de forma uniforme	Módulos con ciclos de cambio muy distintos
Resiliencia requerida	Disponibilidad estándar (99.5%)	Alta disponibilidad crítica (99.9%+)

Otras arquitecturas a considerar

- Monolito Modular — monolito con límites de contexto claros internamente (packages, módulos). El punto medio ideal antes de extraer servicios.
- Serverless / FaaS — funciones ejecutadas bajo demanda (AWS Lambda, Google Cloud Functions). Ideal para cargas variables e imprevisibles.
- JAMstack — JavaScript, APIs y Markup pre-generado. Muy usado para sitios con contenido relativamente estático y bajo tiempo de carga.
- BFF (Backend for Frontend) — un backend dedicado por tipo de cliente (móvil, web, TV). Optimiza las APIs para cada canal de consumo.

> **ESTRATEGIA RECOMENDADA PARA NUEVOS PROYECTOS**
>
> 1. Comienza con un Monolito Modular bien estructurado aplicando principios MVC y separación de contextos.
> 2. Identifica con datos reales de producción los cuellos de botella: ¿qué módulo necesita escalar de forma diferente?

3. Extrae ese módulo como microservicio solo cuando el costo de no hacerlo supera el costo de la complejidad adicional.
4. Repite el proceso de forma incremental. Este enfoque se conoce como 'Strangler Fig Pattern'.

SÍNTESIS DE LA UNIDAD

Fundamentos del Desarrollo Web Backend

- **La web moderna** se estructura en cliente y servidor, comunicados mediante el protocolo HTTP sobre una infraestructura de red TCP/IP.
- **Front End y Back End** son especialidades complementarias: el primero gestiona la experiencia del usuario, el segundo la lógica, los datos y la seguridad.
- **El backend** es el núcleo funcional de cualquier aplicación: sin él, el frontend es solo una interfaz estática.
- **HTTP** define verbos (métodos), metadatos (cabeceras) y resultados (códigos de estado) que estructuran toda comunicación web y son la base del diseño de APIs REST.
- **El Monolito MVC** es el punto de partida natural: sencillo de desarrollar y operar, ideal para equipos pequeños y dominios en exploración.
- **Los Microservicios** ofrecen escalabilidad granular y autonomía de equipos, pero requieren madurez técnica y operativa que justifique la inversión.
- **La decisión arquitectónica** debe basarse en datos del contexto real del proyecto, no en tendencias tecnológicas.